

Linux Kernel Buffer Overflow Vulnerabilities

Analysis Report

January 2026

1 Executive Summary

This document analyzes 19 Linux kernel buffer overflow vulnerabilities from 2021–2025, categorizing them into 7 distinct patterns. The vulnerabilities range from classic stack overflows to more subtle semantic errors such as length field misinterpretation, and allocation/usage count mismatches. Two vulnerabilities (CVE-2024-53104 and CVE-2024-53197) appear in CISA’s Known Exploited Vulnerabilities catalog.

Table 1: Vulnerability Pattern Summary

Pattern	Count	Detection Approach
Unbounded sprintf/memcpy	7	AST pattern matching
Integer Arithmetic Error	4	Value range/overflow analysis
Allocation/Use Mismatch	2	Cross-function analysis
Validate-A-Use-B	2	Variable tracking across validation/use
String Termination	2	strncpy usage patterns
Length Field Mismatch	1	Protocol structure analysis
Stale Constant	1	Enum vs constant cross-reference
Total	19	

2 Case Studies

2.1 1. CVE-2022-0435: TIPC Remote Stack Overflow

File: net/tipc/monitor.c Pattern: Missing Bounds Check

```
1 struct tipc_mon_domain {
2     u32 len;
3     u32 gen;
4     u32 member_cnt;
5     u32 members[MAX_MON_DOMAIN]; // MAX_MON_DOMAIN = 64
6 };
7
8 static void tipc_mon_rcv(struct net *net, void *data, ...)
9 {
10     struct tipc_mon_domain *dom = data;
11     struct tipc_mon_domain dom_bef; // 272 bytes on stack
12
13     // BUG: No check that dom->member_cnt <= MAX_MON_DOMAIN
14     // Attacker controls dom->len
15     memcpy(&dom_bef, dom, dom->len);
16
17     // FIXED: Validate member_cnt and len before copy
18     if (dom->member_cnt > MAX_MON_DOMAIN) return;
19     if (dom->len > sizeof(struct tipc_mon_domain)) return;
20     memcpy(&dom_bef, dom, dom->len);
21 }
```

Analysis: The TIPC protocol allows cluster nodes to exchange domain membership information. The dom_bef structure is sized for 64 members (272 bytes). An attacker sends member_cnt = 264 with len=1072. The memcpy copies 1072 bytes into a 272-byte stack buffer, overwriting saved registers and return address.

2.2 2. CVE-2021-43267: TIPC Remote Heap Overflow

File: net/tipc/crypto.c Pattern: Allocation/Copy Size Mismatch

```
1 static bool tipc_crypto_key_rcv(struct tipc_crypto *rx, struct tipc_msg *hdr)
2 {
3     u16 size = msg_data_sz(hdr); // From packet header
4     u8 *data = msg_data(hdr);
5     struct tipc_aead_key *skey = kmalloc(size, GFP_ATOMIC); // Allocate based on header
6
7     // BUG: keylen from packet body, not validated against size
8     skey->keylen = ntohs(*((__be32*)(data + TIPC_AEAD_ALG_NAME)));
9     memcpy(skey->key, data + offset, skey->keylen); // Uses keylen!
10
11     // FIXED: Validate keylen matches declared packet size
12     if (size < sizeof(struct tipc_aead_key) + TIPC_AEAD_KEYLEN_MIN) goto exit;
13     // ... validation logic ...
14 }
```

Analysis: There are two independent sizes: packet header declares one used for allocation, and the packet body contains keylen which is used for memcpy. The attacker sends small declared size of 32 bytes but large keylen size of 4096 bytes. Kernel allocates 32 bytes, copies 4096 resulting in a classic heap overflow.

2.3 3. CVE-2024-38576: RCU sprintf Stack Overflow

File: kernel/rcu/tree_stall.h Pattern: Unbounded Format String

```
1 static void print_cpu_stall_info(int cpu)
2 {
3     char buf[32];
4     long j;
5     // ...
6     if (rcuc_starved)
7         sprintf(buf, "rcuc=%ld jiffies (starved)", j);
```

```

8 // Max output: 6 + 20 + 17 + 1 = 44 bytes into 32-byte buffer
9
10 // FIXED
11 if (rcuc_starved)
12     snprintf(buf, sizeof(buf), "rcuc=%ld jiffies(starved)", j);
13 }

```

Analysis: `snprintf` writes until format completion regardless of buffer size. The `%ld` specifier produces up to 20 digits for `LONG_MAX`. Combined with fixed strings, max output is 44 bytes into 32-byte buffer.

2.4 4. CVE-2024-38577: RCU-tasks `snprintf` Overflow

File: `kernel/rcu/tasks.h` **Pattern:** Unbounded Format String

```

1 static void show_rcu_tasks_trace_gp_kthread(void)
2 {
3     char buf[64];
4     /* BUG: 4x %lu (20 digits each) + fixed chars exceeds 64 bytes */
5     sprintf(buf, "N%lu h:%lu/%lu/%lu",
6             data_race(n_trc_holdouts),
7             data_race(n_heavy_reader_ofl_updates),
8             data_race(n_heavy_reader_updates),
9             data_race(n_heavy_reader_attempts));
10
11     // Max output: 80 digits + 6 chars + 1 null = 87 bytes
12
13     // FIXED: Enforce buffer size limit
14     snprintf(buf, sizeof(buf), "N%lu h:%lu/%lu/%lu", ...);
15 }

```

Analysis: Four unsigned long values can each produce 20 digits. Combined with the format string prefixes, the maximum output is 87 bytes, which overflows the 64-byte stack buffer

2.5 5. CVE-2024-26733: ARP Hardware Address Overflow

File: `net/ipv4/arp.c` **Pattern:** Fixed Dest, Variable Source

```

1 static int arp_req_get(struct arpreq *r, struct net_device *dev)
2 {
3     struct neighbour *neigh;
4     // r->arp_ha.sa_data is 14 bytes (sockaddr)
5     // neigh->ha can be up to MAX_ADDR_LEN (32 bytes) for InfiniBand
6
7     memcpy(r->arp_ha.sa_data, neigh->ha, dev->addr_len);
8     // If dev->addr_len > 14, overflow into arp_flags field.
9
10    // FIXED
11    memcpy(r->arp_ha.sa_data, neigh->ha,
12           min(dev->addr_len, sizeof(r->arp_ha.sa_data)));
13 }

```

Analysis: `sockaddr`'s `sa_data` is 14 bytes. InfiniBand has 32-byte addresses. The `ioctl` copies neighbor's hardware address without clamping to destination size.

2.6 6. CVE-2024-49996: CIFS Reparse Point Length Mismatch

File: `fs/smb/client/reparse.c` **Pattern:** Length Field Semantic Mismatch

```

1 struct reparse_posix_data {
2     __le16 ReparseDataLength; // Includes InodeType size
3     __le16 ReparseTag;
4     __le64 InodeType;          // 8 bytes
5     u8 DataBuffer[];          // Actual symlink data
6 };
7
8 static int parse_reparse_posix(struct reparse_posix_data *buf, ...)

```

```

9 {
10     int len = le16_to_cpu(buf->ReparseDataLength);
11
12     /* BUG: len includes sizeof(InodeType) (8 bytes).
13        Using it directly as the length for DataBuffer reads
14        8 bytes past the end of the allocated buffer. */
15     // data->symlink_target = cifs_strndup_from_utf16(buf->DataBuffer, len, ...);
16
17     /* FIXED: Validate and subtract header size */
18     if (len < sizeof(buf->InodeType))
19         return -EIO;
20
21     len -= sizeof(buf->InodeType); // Now len is actual DataBuffer size
22 }

```

Analysis: The ReparseDataLength field includes the size of InodeType (8 bytes) plus the DataBuffer. The vulnerable code used the total length for the memory copy. If a server sends a packet where ReparseDataLength equals 100, the DataBuffer is only 92 bytes. The code would read 8 bytes past the end of the buffer into the heap.

2.7 7. CVE-2024-53197: USB Audio Descriptor Mismatch [CISA KEV]

File: sound/usb/quirks.c **Pattern:** Allocation/Use Mismatch

```

1 static int snd_usb_extigy_boot_quirk(struct usb_device *dev, ...)
2 {
3     struct usb_host_config *config = dev->actconfig;
4     // dev->config allocated based on device descriptor's bNumConfigurations
5     // But code uses config descriptor's bNumConfigurations as index
6
7     if (get_cfg_desc(config)->bNumConfigurations != 1) {
8         err = usb_driver_set_configuration(dev, 1);
9     }
10
11     // FIXED: Validate config descriptor value against allocation
12     if (get_cfg_desc(config)->bNumConfigurations > dev->descriptor.bNumConfigurations) {
13         dev_err(&dev->dev, "bogus bNumConfigurations\n");
14         return -EINVAL;
15     }
16 }

```

Analysis: Device descriptor says bNumConfigurations = 2 (allocates config[2]). Config descriptor says bNumConfigurations = 10. Code accesses config[9] 8 past allocation. CISA KEV: Actively exploited.

2.8 8. CVE-2024-53104: UVC Driver Count Mismatch [CISA KEV]

File: drivers/media/usb/uvc/uvc_driver.c **Pattern:** Allocation/Use Mismatch

```

1 // Phase 1: Count frames for allocation
2 for (...) {
3     switch (desc->bDescriptorSubType) {
4         case UVC_VS_FRAME_UNCOMPRESSED:
5         case UVC_VS_FRAME_MJPEG:
6         case UVC_VS_FRAME_FRAME_BASED:
7             nframes++; // Count 3 types
8     }
9 }
10 streaming->frame = kcalloc(nframes, sizeof(*frame), GFP_KERNEL);
11
12 // Phase 2: Parse handles 4 types including UVC_VS_UNDEFINED!
13 switch (...) {
14     case UVC_VS_UNDEFINED:
15         uvc_parse_frame(&streaming->frame[frame_idx++]); // OOB
16 }

```

Analysis: Counting phase handles 3 frame types; parsing handles 4. Malicious USB camera with 3 normal + 1 UNDEFINED frame causes out of bounds write.

2.9 9. CVE-2023-0179: Netfilter Integer Underflow

File: net/netfilter/nft_payload.c Pattern: Operator Typo

```
1 static bool nft_payload_copy_vlan(u32 d, const struct sk_buff *skb, u8 offset, u8 len)
2 {
3     // ...
4     if (skb_vlan_tag_present(skb)) {
5         if (offset + len > VLAN_ETH_HLEN + vlan_hlen)
6             ethlen -= offset + len - VLAN_ETH_HLEN + vlan_hlen;
7         // BUG: + vlan_hlen instead of - vlan_hlen
8
9         // FIXED:
10        ethlen -= offset + len - VLAN_ETH_HLEN - vlan_hlen;
11    }
12 }
```

Analysis: Single character typo: + instead of -. With offset=0, len=40: buggy calc gives ethlen = 22 - 26 = -4 which as u8 = 252. memcpy copies 252 bytes to 16-byte stack buffer. Public PoC achieves LPE.

2.10 10. CVE-2024-38541: snprintf Return Value Misuse

File: drivers/of/module.c Pattern: Integer Arithmetic Error

```
1 ssize_t of_modalias(const struct device_node *np, char *str, ssize_t len)
2 {
3     ssize_t csize;
4     // snprintf returns predicted length
5     csize = snprintf(str, len, "of:N%pOfN%c%s", np, 'T', ...);
6
7     // BUG: If csize (100) >= len (50), len becomes negative.
8     // len -= csize;
9
10    // FIXED: Clamp csize to available space before subtraction
11    if (csize >= len)
12        csize = len > 0 ? len - 1 : 0;
13
14    len -= csize;
15    str += csize;
16
17    of_property_for_each_string(np, "compatible", p, compat) {
18        csize = strlen(compat) + 1;
19        // FIXED: Strict check (>=) prevents one-byte overflow
20        if (csize >= len)
21            continue;
22
23        csize = snprintf(str, len, "%s", compat);
24    }
25 }
```

Analysis: snprintf returns the length of the string that would have been written if the buffer were infinite. If the buffer is 50 bytes but the output is 100 bytes, it returns 100. The original code performed len -= csize (50 - 100), resulting in a negative value. When passed to subsequent functions, this signed negative value is implicitly cast to a large unsigned integer, allowing out-of-bounds writes. The fix clamps the return value to the safe buffer limit.

2.11 11. CVE-2025-68763: StarFive Crypto Signedness Error

File: drivers/crypto/starfive/jh7110-hash.c Pattern: Signed Integer Conversion Error

```
1 static int starfive_hash_digest(struct ahash_request *req)
2 {
3     struct starfive_cryp_request_ctx *rctx = ahash_request_ctx(req);
4
5     /* BUG: 'sg_nents_for_len' returns int (can be negative error).
```

```

6     'rctx->in_sg_len' is unsigned long.
7     If function returns -EINVAL, implicit cast creates HUGE positive value. */
8     // rctx->in_sg_len = sg_nents_for_len(rctx->in_sg, rctx->total);
9
10    /* FIXED: Capture return value in signed int and check for error first */
11    int sg_len = sg_nents_for_len(rctx->in_sg, rctx->total);
12
13    if (sg_len < 0)
14        return sg_len; /* Return the negative error code properly */
15
16    rctx->in_sg_len = sg_len; /* Safe assignment now that we know it is positive */
17
18    return crypto_transfer_hash_request_to_engine(cryp->engine, req);
19 }

```

Analysis: The function `sg_nents_for_len` returns an `int`, allowing negative error codes such as `-EINVAL`. When assigned directly to an unsigned variable, these negative values are cast to massive positive integers. The driver incorrectly trusts this huge value as the number of memory entries, leading to out-of-bounds access.

2.12 12. CVE-2025-37979: Stale Array Size Constant

File: `sound/soc/qcom/sc7280.c` **Pattern:** Stale Constant

```

1 #define LPASS_MAX_PORTS 8 // Original value
2 // New commit added: RX_CODEC_DMA_RX_0 = 16
3
4 static int sc7280_snd_hw_params(...) {
5     static struct port_config configs[LPASS_MAX_PORTS]; // [8]
6     switch (port_id) {
7         case RX_CODEC_DMA_RX_0: // Value is 16
8             configs[port_id] = new_config; // 00B configs[16]
9     }
10 }
11 // FIXED:
12 #define LPASS_MAX_PORTS 32

```

Analysis: Commit added new port ID enum values (up to 16) without updating `LPASS_MAX_PORTS` from 8. Classic pattern: enum expands but sizing constant not updated.

2.13 13. CVE-2025-38676: strncpy Missing Null Terminator

File: `drivers/iommu/amd/init.c` **Pattern:** String Termination Failure

```

1 static int __init parse_ivrs_acpihid(char *str) {
2     char acpiid[ACPIID_LEN];
3     // If strlen(str) >= ACPIID_LEN, no null terminator!
4     strncpy(acpiid, str, ACPIID_LEN);
5     process_acpi_id(acpiid); // strlen() reads past buffer
6
7     // FIXED: Explicit null termination
8     strncpy(acpiid, str, ACPIID_LEN - 1);
9     acpiid[ACPIID_LEN - 1] = '\0';
10 }

```

Analysis: `strncpy` does NOT null-terminate if source $\geq n$ bytes. Subsequent string operations read past buffer until finding null byte in adjacent memory.

2.14 14. CVE-2025-40004: net/9p USB Gadget Buffer Overflow

File: `net/9p/trans_usb.c` **Pattern:** Validate A Use B

```

1 static void usb9pfs_rx_complete(struct usb_ep *ep, struct usb_request *req)
2 {
3     struct p9_req_t *p9_rx_req = ...; // Get 9p request from queue
4 }

```

```

5 // BUG: Uses req->actual from USB transfer without validation.
6 // p9_rx_req->rc.capacity is the allocated buffer size.
7 // memcpy(p9_rx_req->rc.sdata, req->buf, req->actual);
8
9 // FIXED: Validate actual transfer size against buffer capacity
10 unsigned int req_size = req->actual;
11
12 if (req_size > p9_rx_req->rc.capacity) {
13     req_size = 0; // Prevent overflow
14     // Handle error...
15 }
16
17 memcpy(p9_rx_req->rc.sdata, req->buf, req_size);
18 }

```

Analysis: The driver separates the definition of the buffer capacity from the actual data received via USB. The vulnerability occurs because the code trusts the USB hardware's reported transfer length `req->actual` without verifying it fits into the 9p reply buffer

2.15 15. CVE-2025-39988 CAN etas es58x: Missing MTU Handler

File: `drivers/net/can/usb/etas_es58x/es58x_core.c` **Pattern:** Missing Validation

```

1 static const struct net_device_ops es58x_netdev_ops = {
2     .ndo_start_xmit = es58x_start_xmit,
3     // Missing: ndo_change_mtu allows ANY MTU
4 };
5
6 static netdev_tx_t es58x_start_xmit(struct sk_buff *skb, ...) {
7     // Driver expects CAN FD (64 bytes max)
8     // CAN XL frame with len 2048 causes overflow
9     memcpy(tx_can_msg->data, cf->data, cf->len);
10 }
11
12 // FIXED: Add ndo_change_mtu handler
13 static int es58x_change_mtu(struct net_device *netdev, int new_mtu) {
14     if (new_mtu > CAN_MTU && new_mtu > CANFD_MTU) return -EINVAL;
15     netdev->mtu = new_mtu;
16     return 0;
17 }

```

Analysis: Without `ndo_change_mtu`, default handler accepts any MTU. CAN XL frames can have 2048-byte payloads. Driver expects CAN FD (64 max) and overflows.

2.16 16. CVE-2025-68753: ALSA Firewire Bounds Check

File: `sound/firewire/motu/motu-hwdep.c` **Pattern:** Missing Bounds Check

```

1 static long hwdep_read(struct snd_hwdep *hwdep, char __user *buf, long count, loff_t *offset)
2 {
3     int consumed = 0;
4     u32 ev; /* Event data (4 bytes) */
5     u32 __user *ptr;
6
7     /* Loop while ANY space remains in buffer */
8     while (consumed < count && get_next_event(&ev)) {
9         ptr = (u32 __user *) (buf + consumed);
10
11         /* BUG: Loop guard (consumed < count) allows entry with 1 byte left,
12            but put_user writes 4 bytes. Overwrites user stack. */
13         // if (put_user(ev, ptr))
14
15         /* FIXED: Check if the FULL write fits within the user's count */
16         if (consumed + sizeof(ev) > count || put_user(ev, ptr))
17             return -EFAULT;
18
19         consumed += sizeof(ev);

```

```

20 }
21     return consumed;
22 }

```

Analysis: In the dsp event handling code, the `put_user` loop copies event data, however when the buffer size is not 4 byte aligned it could overwrite past the buffer boundary. Fix by adding bounds check before `put_user`

2.17 17. CVE-2024-27045: snprintf Hardcoded Size Mismatch

File: `drivers/gpu/drm/amd/display/amdgpu_dm/amdgpu_dm_debugfs.c` **Pattern:** Incorrect Size Parameter

```

1 static ssize_t dp_dsc_clock_en_read(...) {
2     char rd_buf[10];
3
4     // BUG: 30 > sizeof(rd_buf)
5     snprintf(rd_buf, 30, "%d\n", dsc_clock_en);
6
7     // FIXED:
8     snprintf(rd_buf, rd_buf_size, "%d\n", dsc_clock_en);
9 }

```

Analysis: The buffer is 10 bytes, while the size hardcoded in `snprintf` is 30. The defined buffer limit `rd_buf` should be used in `sprintf`.

2.18 18. CVE-2025-39998 SCSI target_core_configs: snprintf Return Unchecked

File: `drivers/target/target_core_configs.c` **Pattern:** snprintf Return Misuse

```

1 static ssize_t target_lu_gp_members_show(...) {
2     char buf[LU_GROUP_NAME_BUF]; // 256 bytes
3     int written;
4
5     written = snprintf(buf, sizeof(buf), "%s/%s\n", ...);
6
7     // BUG: written can exceed buf
8     memcpy(page + len, buf, written);
9
10    // FIXED:
11    if (written >= sizeof(buf)) written = sizeof(buf) - 1;
12 }

```

Analysis: `snprintf` returns bytes that would be written. If path > 255 chars, returns 300 but `buf` contains only 255. `memcpy` reads 300 from 256-byte buffer.

2.19 CVE-2024-39480: KDB Tab-Complete Buffer Overflow

File: `kernel/debug/kdb/kdb_io.c`

Pattern: `strncpy` Size Mismatch

```

1 static int kdb_read(char *buffer, size_t buf_size)
2 {
3     char *cp = buffer + len; // Cursor position
4     char *lastchar = buffer + count; // End of current data
5     char *bufend = buffer + buf_size; // End of allocated buffer
6     char *p_tmp = ...; // Completion candidate
7
8     // BUG: copy based on source buffer length
9     // strncpy(cp, p_tmp+len, strlen(p_tmp) - len);
10
11    int len_tmp = strlen(p_tmp) - len; // Calculate characters to add
12
13    // 1. Check if adding len_tmp puts us past bufend
14    if (lastchar + len_tmp >= bufend)
15        len_tmp = bufend - lastchar; // Truncate to fit exactly
16

```



```

17     if (len_tmp > 0) {
18         // +1 to account for null terminator
19         memmove(cp + len_tmp, cp, (lastchar - cp) + 1);
20
21         //copy new part of completion string
22         memcpy(cp, p_tmp + len, len_tmp);
23
24         cp += len_tmp;
25         lastchar += len_tmp;
26     }
27 }

```

Analysis: The kernel debugger tab completion logic contained a critical API misuse. The vulnerable code passed the source string's length to `strncpy` as the limit, rather than the remaining space in the destination buffer. If the command buffer was near its limit of 256 bytes, this allowed writing past the `bufend` pointer, corrupting adjacent stack memory. The fix replaces `strncpy` with an explicit remaining space calculation and uses `memmove/memcpy`.

3 Pattern Summary for Detection

3 Pattern Summary for Detection

This section summarizes the seven distinct vulnerability patterns identified across the 19 analyzed CVEs, along with recommended detection strategies.

- (a) **Unbounded Copy to Fixed Buffer (Count: 7)**
Detection Strategy: AST pattern matching.
Identify `sprintf`, `strcpy`, or `memcpy` calls where the destination is a fixed-size buffer and the source length is unbounded or derived from external input without a preceding check.
CVEs: 2024-38576, 2024-38577, 2024-26733, 2025-39988, 2024-39480, 2022-0435, 2025-68753 .
- (b) **Length Field Semantic Mismatch (Count: 1)**
Detection Strategy: Protocol structure analysis.
For protocol structures containing length fields and flexible array members, verify that the length used for memory copying is adjusted by the header size. Requires protocol-specific knowledge or heuristics to identify packet fields.
CVE: 2024-49996.
- (c) **Allocation/Usage Count Mismatch (Count: 2)**
Detection Strategy: Cross-function analysis.
Compare enumeration logic between allocation and access sites (e.g., array index sources).
CVEs: 2024-53104, 2024-53197.
- (d) **Integer Arithmetic Errors (Count: 4)**
Detection Strategy: Value range/overflow analysis.
Track unsigned subtractions or additions without prior validation, and signed to unsigned conversions. Flag `snprintf` return values used in arithmetic and implicit casts of negative error codes to unsigned sizes.
CVEs: 2023-0179, 2024-38541, 2025-39998, 2025-68763.
- (e) **Stale Constants (Count: 1)**
Detection Strategy: Enum vs Constant cross-reference.
Cross-reference array size constants against maximum enumeration values used as indices. This pattern occurs when enums expand but the corresponding sizing constant is not updated.
CVE: 2025-37979.
- (f) **Validate-A-Use-B (Count: 2)**
Detection Strategy: Variable tracking across validation/use.
Track which specific variable is validated against a limit versus which variable is used as the length argument in `memcpy`.
CVEs: 2025-40004, 2021-43267.
- (g) **String Termination Failures (Count: 2)**
Detection Strategy: `strncpy` usage patterns.
Identify `strncpy` calls where the length argument equals the destination size (`sizeof(dest)`). Flag if no explicit null termination follows immediately. Also checks for `snprintf` size parameters matching the buffer size without accounting for the null terminator. In both cases actual size vs size allocated differ by 1.
CVEs: 2025-38676, 2024-27045.

4 4. References

1. **CVE-2022-0435:** *TIPC: Remote Stack Overflow in Monitor Protocol*. National Vulnerability Database, 2022.
2. **CVE-2021-43267:** *TIPC: Remote Heap Overflow in Crypto Key Reception*. Linux Kernel Mailing List, 2021.
3. **CVE-2024-38576:** *RCU: Stack Overflow in print_cpu_stall_info*. Linux Kernel Git (Commit b35fd5d), 2024.
4. **CVE-2024-38577:** *RCU-tasks: Stack Overflow in show_rcu_tasks_trace_gp_kthread*. Linux Kernel Git, 2024.
5. **CVE-2024-26733:** *IPv4/ARP: Stack Overflow in arp_req_get*. Linux Kernel Git, 2024.
6. **CVE-2024-49996:** *SMB Client: Heap Overflow in Reparse Point Parsing*. Linux Kernel Git, 2024.
7. **CVE-2024-53197:** *USB Audio: OOB Access in Extigy Boot Quirk*. CISA Known Exploited Vulnerabilities Catalog, 2024.
8. **CVE-2024-53104:** *USB Video: OOB Write in UVC Frame Parsing*. CISA Known Exploited Vulnerabilities Catalog, 2024.
9. **CVE-2023-0179:** *Netfilter: Integer Underflow in VLAN Payload Copy*. Linux Kernel Git, 2023.
10. **CVE-2024-38541:** *Device Tree: Buffer Overflow via Integer Misinterpretation*. Linux Kernel Git, 2024.
11. **CVE-2025-68763:** *StarFive Crypto: Signedness Error in Hash Digest*. Linux Kernel Git, 2025.
12. **CVE-2025-37979:** *ASoC: OOB Access via Stale Port Constant*. Linux Kernel Git, 2025.
13. **CVE-2025-38676:** *AMD IOMMU: Stack Overflow via Missing Null Terminator*. Linux Kernel Git, 2025.
14. **CVE-2025-40004:** *9P Gadget: Heap Overflow via USB Transfer Size Mismatch*. Linux Kernel Git, 2025.
15. **CVE-2025-39988:** *CAN: Heap Overflow via Missing MTU Handler*. Linux Kernel Git, 2025.
16. **CVE-2025-68753:** *ALSA Firewire: Stack Overflow in DSP Event Loop*. Linux Kernel Git, 2025.
17. **CVE-2024-27045:** *AMDGPU: Buffer Overflow in Display Manager DebugFS*. Linux Kernel Git, 2024.
18. **CVE-2025-39998:** *SCSI Target: Stack Info Leak via snprintf*. Linux Kernel Git, 2025.
19. **CVE-2024-39480:** *KDB: Stack Overflow in Tab Completion*. Linux Kernel Git, 2024.